План курса

Часть 1. Объектно-ориентированное программирование

Тем: 3

Тема 1. Программирование по контракту

Содержание

1. Контракты кода
    1. Пред- и постусловия
    2. Операторы присваивания, композиции, ветвления, цикла
    3. Стратегии доказательств
2. Контракты функций
    1. Чистые функции
    2. Хранимое состояние и инварианты

Слайды

HTML

Примеры

zip

Домашнее задание 1. Обработка ошибок

1. Разработайте классы `Checked*`, выполняющие соответствующие операции с проверкой на переполнения и неверные аргументы.
2. Добавьте в программу, вычисляющую выражения, обработку ошибок, в том числе:
    - ошибки разбора выражений;
    - ошибки вычисления выражений.
3. Для выражения `1000000*x*x*x*x*x/(x-1)` вывод программы должен иметь следующий вид:

```
x       f
0       0
1       division by zero
2       32000000
3       121500000
4       341333333
5       overflow
6       overflow
7       overflow
8       overflow
9       overflow
10      overflow
```

   Результат `division by zero` (`overflow`) означает, что в процессе вычисления произошло деление на ноль (переполнение).
4. При выполнении задания следует обратить внимание на дизайн и обработку исключений.
5. Человеко-читаемые сообщения об ошибках должны выводиться на консоль.
6. Программа не должна «падать» с исключениями (как стандартными, так и добавленными).
7. Код должен находиться в пакете `expression.exceptions`.

Репозиторий курса

Домашнее задание 2. Бинарный поиск

1. Реализуйте итеративный и рекурсивный варианты бинарного поиска в массиве.
2. На вход подается целое число `x` и массив целых чисел `a`, отсортированный по невозрастанию. Требуется найти минимальное значение индекса `i`, при котором `a[i] ≤ x`.
3. Для `main`, функций бинарного поиска и вспомогательных функций должны быть указаны, пред- и постусловия. Для реализаций методов должны быть приведены доказательства соблюдения контрактов в терминах троек Хоара.
4. Интерфейс программы.
    - Имя основного класса — `search.BinarySearch`.
    - Первый аргумент командной строки — число `x`.
    - Последующие аргументы командной строки — элементы массива `a`.
5. Пример запуска: `java search.BinarySearch 3 5 4 3 2 1`. Ожидаемый результат: `2`.

Библиография

1. *Дейкстра Э.* Дисциплина программирования
2. *Мейер Б.* Объектно-ориентированное конструирование программных систем
3. *Hoare C.* An axiomatic basis for computer programming

Тема 2. Реализация ООП

Содержание

1. Классы
    1. Инвариант класса
    2. Задачи инкапсуляции
2. Интерфейсы
    1. Интерфейс как синтаксический контракт
    2. Интерфейс как семантический контракт
3. Абстрактные базовые классы и наследование
    1. Устранение дублирования
    2. Вынос изменяемой логики в наследников

Слайды

HTML

Домашнее задание 3. Очередь на массиве

1. Определите модель и найдите инвариант структуры данных «очередь».
    - Определите функции, которые необходимы для реализации очереди.
    - Найдите их пред- и постусловия, если очередь не может содержать `null`.
2. Реализуйте классы, представляющие **циклическую** очередь на основе массива.
    - Класс `ArrayQueueModule` должен реализовывать один экземпляр очереди с использованием переменных класса.
    - Класс `ArrayQueueADT` должен реализовывать очередь в виде абстрактного типа данных (с явной передачей ссылки на экземпляр очереди).
    - Класс `ArrayQueue` должен реализовывать очередь в виде класса (с неявной передачей ссылки на экземпляр очереди).
    - Должны быть реализованы следующие функции (процедуры) / методы:
        - `enqueue` – добавить элемент в очередь;
        - `element` – первый элемент в очереди;
        - `dequeue` – удалить и вернуть первый элемент в очереди;
        - `size` – текущий размер очереди;
        - `isEmpty` – является ли очередь пустой;
        - `clear` – удалить все элементы из очереди.
    - Модель, инвариант, пред- и постусловия записываются в исходном коде в виде комментариев.
    - Обратите внимание на инкапсуляцию данных и кода во всех трех реализациях.
3. Напишите простые тесты к реализованным классам.

Домашнее задание 4. Очереди

1. Определите интерфейс очереди `Queue` и опишите его контракт.
2. Реализуйте класс `LinkedQueue` — очередь на связном списке.
3. Выделите общие части классов `LinkedQueue` и `ArrayQueue` в базовый класс `AbstractQueue`.

Это домашнее задание *связано* с предыдущим.

Примеры

zip

Экзаменационные вопросы

1. Пре- и постусловия, инварианты
2. Интерфейсы
3. Абстрактные базовые классы
4. Принцип подстановки Лисков

Практические навыки

1. Реализация программ на Java с применением интерфейсов и абстрактных базовых классов

Тема 3. Generics

Содержание

1. Введение в Generics
    1. Полиморфизм и его типы
    2. Generics и templates
    3. Применение Generics
2. Реализация Generics
    1. Wildcards и ограниченные wildcards
    2. Параметры типов и ограниченные параметры типов
    3. Ограничения Generics и reifables
3. Generics и вариантность
    1. Ко- и контрвариантность
    2. Источники и приемники
    3. Правило выбора extends и super
4. Смешение Generic и не Generic кода
    1. Стирание информации о типе и сырые типы
    2. Добавление информации о типе и UncheckedWarning

Слайды

HTML

Домашнее задание 5. Вычисление в различных типах

Добавьте в программу разбирающую и вычисляющую выражения трех переменных поддержку вычисления в различных типах.

1. Создайте класс `expression.generic.GenericTabulator`, реализующий интерфейс `expression.generic.Tabulator`:

```
public interface Tabulator {
    Object[][][] tabulate(
        String mode, String expression,
        int x1, int x2, int y1, int y2, int z1, int z2
    ) throws Exception;
}
```

   Аргументы

    - `mode` — режим работы

Режим	Тип
`i`	`int` с детекцией переполнений
`d`	`double`
`bi`	`BigInteger`

    - `expression` — вычисляемое выражение;
    - `x1`, `x2`; `y1`, `y2`; `z1`, `z2` — диапазоны изменения переменных (включительно).

   Возвращаемое значение — таблица значений функции, где `R[i][j][k]` соответствует `x = x1 + i`, `y = y1 + j`, `z = z1 + k`. Если вычисление завершилось ошибкой, в соответствующей ячейке должен быть `null`.

2. Доработайте интерфейс командной строки:
    - Первым аргументом командной строки программа должна принимать указание на тип, в котором будут производиться вычисления:

Опция	Тип
`-i`	`int` с детекцией переполнений
`-d`	`double`
`-bi`	`BigInteger`

    - Вторым аргументом командной строки программа должна принимать выражение для вычисления.
    - Программа должна выводить результаты вычисления для всех целочисленных значений переменных из диапазона −2..2.
3. Реализация не должна содержать непроверяемых преобразований типов.
4. Реализация не должна использовать аннотацию `@SuppressWarnings`.
5. При выполнении задания следует обратить внимание на простоту добавления новых типов и операций.

Библиография

1. Generics Tutorial
2. Generics in the Java Programming Language
3. JSR 14. Generics

Часть 2. Введение в JavaScript

Тем: 3

Тема 4. Функции

Содержание

1. JavaScript и Java
    1. Появление и история JavaScript/ECMAScript
    2. Связь с Java
    3. Различия с Java
    4. Strict mode
2. Переменные и типы
    1. Объявление переменных
    2. Типы значений
    3. Массивы
3. Простые функции
    1. Синтаксис объявления
    2. Передача аргументов
    3. Способы вызова
    4. Стрелочные функции
4. Функции высших порядков
    1. Примеры
        1. Сортировка
        2. Компараторы
        3. Составные компараторы
        4. Функциональные интерфейсы
    2. Комбинаторы
        1. foldLeft, foldRight
        2. map
        3. Композиция
        4. Карринг

Домашнее задание 6. Функциональные выражения на JavaScript

1. Разработайте функции `cnst`, `variable`, `add`, `subtract`, `multiply`, `divide`, `negate` для вычисления выражений с переменной `x`.
2. Функции должны позволять производить вычисления вида:

```
let expr = subtract(
    multiply(
        cnst(2),
        variable("x")
    ),
    cnst(3)
);

println(expr(5));
```

   При вычислении выражения вместо переменной `x` подставляется значение, переданное в качестве аргумента функции `expr`. Таким образом, результатом вычисления приведенного примера должно быть число 7.
3. Тестовая программа должна вычислять выражение x^2-2x+1, для `x` от 0 до 10.
4. **Сложный вариант.** Требуется дополнительно написать функцию `parse`, осуществляющую разбор выражений, записанных в обратной польской записи. Например, результатом

```
parse("x x 2 - * x * 1 +")(5)
```

   должно быть число `76`.
5. При выполнении задания следует обратить внимание на:
    - Применение функций высшего порядка.
    - Выделение общего кода для операций.

Библиография

1. ECMA-262 ECMAScript 2024 Language Specification
2. MDN JavaScript
3. Современный учебник JavaScript
4. *Д. Флэнаган* JavaScript. Подробное руководство, 6 издание

Тема 5. Объекты и замыкания

Содержание

1. Объекты
    1. Объекты как ассоциативные массивы
        1. Свойства
        2. Изменение свойств
        3. Неопределенные свойства
        4. Строки как индексы
        5. Сокращенная запись
        6. Проверка наличия свойства
        7. Перечисление свойств
    2. Наследование
        1. Создание объектов (Object.create)
        2. Получение прототипа (Object.getPrototypeOf)
        3. Наследование свойств
        4. Переопределение свойств
        5. Удаление свойств
    3. Методы
        1. Функции в свойствах
        2. Наследование функций
        3. Aliasing
        4. Неявная передача this
        5. Явная передача this
    4. Конструкторы
        1. Фабрики
        2. Конструкторы
        3. Прототипы в конструкторах
        4. Явное создание объекта и вызов конструктора
        5. Получение конструктора
2. Замыкания
    1. Хранение состояния в замыкании
    2. Замыкание изменяемых переменных
    3. Трюк с объявлением промежуточной функции
    4. Общее состояние нескольких функций
3. Модули
    1. Определение модуля
        1. Определение конструкторов
        2. Определение функций
    2. Использования модуля
        1. Прямое использование
        2. Импортирование
        3. Переменные класса

Домашнее задание 7. Объектные выражения на JavaScript

1. Разработайте классы `Const`, `Variable`, `Add`, `Subtract`, `Multiply`, `Divide`, `Negate` для представления выражений с тремя переменными: `x`, `y` и `z`.
    1. Пример описания выражения `2x-3`:

```
let expr = new Subtract(
    new Multiply(
        new Const(2),
        new Variable("x")
    ),
    new Const(3)
);

println(expr.evaluate(5, 0, 0));
```

    2. При вычислении такого выражения вместо каждой переменной подставляется её значение, переданное в качестве аргумента метода `evaluate`. Таким образом, результатом вычисления приведенного примера должно стать число 7.
    3. Метод `toString()` должен выдавать запись выражения в обратной польской записи. Например, `expr.toString()` должен выдавать «`2 x * 3 -`».
2. Функция `parse` должна осуществлять разбор выражений, записанных в обратной польской записи. Например, результатом

```
parse("x x 2 - * x * 1 +").evaluate(5, 0, 0)
```

   должно быть число `76`, а результатом

```
parse("x x 2 - * x * 1 +").toString()
```

   — строка «`x x 2 - * x * 1 +`».
3. **Сложный вариант.**
   Метод `diff("x")` должен возвращать выражение, представляющее производную исходного выражения по переменной `x`. Например, `expr.diff("x")` должен возвращать выражение, эквивалентное `new Const(2)`. Выражения `new Subtract(new Const(2), new Const(0))` и

```
new Subtract(
    new Add(
        new Multiply(new Const(0), new Variable("x")),
        new Multiply(new Const(2), new Const(1))
    )
    new Const(0)
)
```

   так же будут считаться правильным ответом.
4. **Бонусный вариант.**
   Требуется написать метод `simplify()`, производящий вычисления константных выражений. Например,

```
parse("x x 2 - * 1 +").diff("x").simplify().toString()
```

   должно возвращать «`x x 2 - +`» или аналогичное по сложности эквивалентное выражение.
5. При выполнении задания следует обратить внимание на:
    - Применение инкапсуляции.
    - Выделение общего кода для операций.
    - Минимизацию необходимой памяти.

Библиография

1. ECMA-262 ECMAScript 2018 Language Specification
2. Современный учебник JavaScript

Тема 6. Что еще есть в JavaScript

Содержание

1. Обработка ошибок
    1. Стандартные ошибки
    2. Try-catch-finally
    3. Что можно бросить
    4. Пользовательские исключения
2. Нет такой вещи как...
    1. Глобальные переменные
    2. Предопределенные значения
    3. Блочные переменные
    4. Реализация замыканий
3. Стандартная библиотека
    1. Основные типы
    2. Основные функции
        1. eval
        2. parseInt/Float
        3. isNaN/Finite
    3. Составные типы
        1. Объекты
        2. Функции
        3. Массивы
        4. Строки
        5. Boolean
        6. Number
        7. Date
        8. RegExp
    4. Встроенные ошибки
        1. Error
        2. RangeError
        3. ReferenceError
        4. SyntaxError
        5. TypeError
        6. URIError
    5. Наборы функций
        1. Math
        2. JSON
4. Классы и методы
    1. Свойства-методы
    2. Getters и setters
    3. Классы
    4. Наследование
5. Javascript 6+
    1. Шаблонные строки
    2. Set и Map
    3. Модули
6. Свойства (не рассказывалось)
    1. Определение свойств
    2. Атрибуты свойств: enumerable, writable, configurable
    3. Атрибуты свойств: value, get, set
    4. Атрибуты объектов: extensible, seal/isSealed, freeze/isFrozen

Домашнее задание 8. Обработка ошибок на JavaScript

1. Добавьте в предыдущее домашнее задание функцию `parsePrefix(string)`, разбирающую выражения, задаваемые записью вида «`(- (* 2 x) 3)`». Если разбираемое выражение некорректно, метод `parsePrefix` должен бросать ошибки с человеко-читаемыми сообщениями.
2. Добавьте в предыдущее домашнее задание метод `prefix()`, выдающий выражение в формате, ожидаемом функцией `parsePrefix`.
3. При выполнении задания следует обратить внимание на:
    - Применение инкапсуляции.
    - Выделение общего кода для операций.
    - Минимизацию необходимой памяти.
    - Обработку ошибок.

Библиография

1. ECMA-262 ECMAScript Language Specification 5.1 edition
2. ECMAScript 6 — New Features: Overview & Comparison
3. Современный учебник JavaScript

Часть 3. Введение в Clojure

Тем: 5

Тема 7. Функции

Содержание

1. Выражения и переменные
    1. Константы
    2. Функции
        1. Фиксированной арности
        2. Произвольной арности
    3. Переменные
        1. Определение
        2. Функции как значения
    4. Примитивные типы
    5. Приведение типов
2. Функции
    1. Определение функции
    2. Рекурсивные функции
        1. Общая рекурсия
        2. Хвостовая рекурсия
        3. Оптимизация хвостовой рекурсии
    3. Контракты
3. Списки
    1. Литералы
    2. Создание
    3. Операции
    4. Проверки
    5. Свертки
4. Вектора
    1. Литералы
    2. Создание
    3. Операции
    4. Проверки
    5. Свертки
5. Стандартные функции

Домашнее задание 9. Линейная алгебра на Clojure

1. Разработайте функции для работы с объектами линейной алгебры, которые представляются следующим образом:
    - скаляры – числа
    - векторы – векторы чисел;
    - матрицы – векторы векторов чисел.
2. Функции над векторами:
    - `v+`/`v-`/`v*`/`vd` – покоординатное сложение/вычитание/умножение/деление;
    - `scalar`/`vect` – скалярное/векторное произведение;
    - `v*s` – умножение на скаляр.
3. Функции над матрицами:
    - `m+`/`m-`/`m*`/`md` – поэлементное сложение/вычитание/умножение/деление;
    - `m*s` – умножение на скаляр;
    - `m*v` – умножение на вектор;
    - `m*m` – матричное умножение;
    - `transpose` – транспонирование;
4. **Сложный вариант.**
    1. Ко всем функциям должны быть указаны контракты. Например, нельзя складывать вектора разной длины.
    2. Все функции, кроме `transpose`, должны поддерживать произвольное число аргументов. Например `(v+ [1 2] [3 4] [5 6])` должно быть равно `[9 12]`.
5. При выполнении задания следует обратить внимание на:
    - Применение функций высшего порядка.
    - Выделение общего кода для операций.

Code Golf (бонус)

Правила

1. Выигрывает самая короткая программа. Длина программы считается после удаления незначимых пробелов.
2. Можно использовать произвольные функции стандартной библиотеки Clojure.
3. Нельзя использовать функции Java и внешних библиотек.
4. Подача решений через чат. Решение должно быть корректно отформатировано и начинаться с `;Solution номинация длина`. Например, `;Solution det-3 1000`.

Номинации

- `det-3` — определитель матрицы за $O(n^3)$;
- `det-s` — определитель дольше чем за $O(n^3)$;
- `inv-3` — обратная матрица за $O(n^3)$;
- `inv-s` — обратная дольше чем за $O(n^3)$.

Библиография

1. Clojure
2. Clojure Cheat Sheet
3. Clojure Documentation
4. *D. Higginbotham* Clojure for the Brave and True
5. Введение в Clojure
6. Cursive: Clojure(Script) IDE

Тема 8. Внешний мир

Содержание

1. Ввод-вывод
    1. Простейший ввод-вывод
    2. Работа с последовательностями событий
    3. Разбор ввода
2. Разбор и гомоиконность
    1. Стандартный парсер
    2. Операции с кодом
    3. Исполнение сгенерированного кода
    4. Гомоиконность
3. Подрядки вычисления
    1. Аппликативный
    2. Нормальный
    3. Ленивый
4. Потоки
    1. Основные определения
    2. Операции с потоками
    3. Конечные потоки
    4. Бесконечные потоки
    5. Ленивые последовательности
5. Отображения и множества
    1. Литералы
    2. Создание
    3. Проверки
    4. Операции

Домашнее задание 10. Функциональные выражения на Clojure

1. Разработайте функции `constant`, `variable`, `add`, `subtract`, `multiply`, `divide` и `negate` для представления арифметических выражений.
    1. Пример описания выражения `2x-3`:

```
(def expr
  (subtract
    (multiply
      (constant 2)
      (variable "x"))
    (constant 3)))
```

    2. Выражение должно быть функцией, возвращающей значение выражения при подстановке переменных, заданных отображением. Например, `(expr {"x" 2})` должно быть равно 1.
2. Разработайте разборщик выражений, читающий выражения в стандартной для Clojure форме. Например,

```
(parseFunction "(- (* 2 x) 3)")
```

   должно быть эквивалентно `expr`.
3. **Сложный вариант.** Функции `add`, `subtract`, `multiply` и `divide` должны принимать произвольное число аргументов. Разборщик так же должен допускать произвольное число аргументов для `+`, `-`, `*`, `/`.
4. При выполнении задания следует обратить внимание на:
    - Выделение общего кода для операций.

Библиография

1. Clojure
2. Clojure Cheat Sheet
3. Введение в Clojure

Тема 9. Объекты

Содержание

1. JS-подобные объекты
    1. Ассоциативные массивы и прототипы
    2. Свойства
    3. Методы
    4. Конструкторы
    5. Наследование
2. Java-объекты
    1. Интерфейсы
    2. Реализация
    3. Изменяемые поля
    4. Работа с Java-классами
3. Изменяемое состояние
    1. Динамические переменные
    2. Локальные переменные
    3. Ссылки

Домашнее задание 11. Объектные выражения на Clojure

1. Разработайте конструкторы `Constant`, `Variable`, `Add`, `Subtract`, `Multiply`, `Divide` и `Negate` для представления арифметических выражений.
    1. Пример описания выражения `2x-3`:

```
(def expr
  (Subtract
    (Multiply
      (Constant 2)
      (Variable "x"))
    (Constant 3)))
```

    2. Функция `(evaluate expression vars)` должна производить вычисление выражения `expression` для значений переменных, заданных отображением `vars`. Например, `(evaluate expr {"x" 2})` должно быть равно 1.
    3. Функция `(toString expression)` должна выдавать запись выражения в стандартной для Clojure форме.
    4. Функция `(parseObject "expression")` должна разбирать выражения, записанные в стандартной для Clojure форме. Например,

```
(parseObject "(- (* 2 x) 3)")
```

       должно быть эквивалентно `expr`.
2. **Сложный вариант.**
    1. Конструкторы `Add`, `Subtract`, `Multiply` и `Divide` должны принимать произвольное число аргументов. Парсер так же должен допускать произвольное число аргументов для `+`, `-`, `*`, `/`.
    2. Функция `(diff expression "variable")` должна возвращать выражение, представляющее производную исходного выражения по заданной переменной. Например, `(diff expression "x")` должен возвращать выражение, эквивалентное `(Constant 2)`, при этом выражения `(Subtract (Constant 2) (Constant 0))` и

```
(Subtract
  (Add
    (Multiply (Constant 0) (Variable "x"))
    (Multiply (Constant 2) (Constant 1)))
  (Constant 0))
```

       так же будут считаться правильным ответом.
3. При выполнении задания можно использовать любой способ представления объектов.
4. При выполнении задания можно использовать функции, для определения JS-like объектов, приведённые на лекции.

Библиография

1. Clojure
2. Clojure Cheat Sheet
3. *Daniel Higginbotham* Clojure for the Brave and True (Chapters 10, 12)

Тема 10. Комбинаторные парсеры

Содержание

1. Базовые концепции
    1. Результат разбора
    2. Интерфейс парсера
    3. Простейшие парсеры
    4. Простейшие комбинаторы парсеров
2. Комбинаторы
    1. Вспомогательные методы
    2. Последовательности
    3. Альтернативы
    4. Регулярные замыкания
3. Пример: JSON
    1. Простые парсеры
    2. Массивы
    3. Объекты
    4. Полный парсер

Домашнее задание 12. Комбинаторные парсеры

1. **Простой вариант.** Реализуйте функцию `(parseObjectPostfix "expression")`, разбирающую выражения, записанные в постфиксной форме, и функцию `toStringPostfix`, возвращающую строковое представление выражения в этой форме. Например,

```
(toStringPostfix (parseObjectPostfix "( ( 2 x * ) 3 - )"))
```

   должно возвращать `((2 x *) 3 -)`.
2. **Сложный вариант.** Реализуйте функцию `(parseObjectInfix "expression")`, разбирающую выражения, записанные в инфиксной форме, и функцию `toStringInfix`, возвращающую строковое представление выражения в этой форме. Например,

```
(toStringInfix (parseObjectInfix "2 * x - 3"))
```

   должно возвращать `((2 * x) - 3)`.
3. **Бонусный вариант.** Добавьте в библиотеку комбинаторов возможность обработки ошибок и продемонстрируйте ее использование в вашем парсере.
4. Функции разбора должны базироваться на библиотеке комбинаторов, разработанной на лекции.

Библиография

1. Clojure
2. Clojure Cheat Sheet
3. Введение в Clojure

Тема 11. Макросы и основания информатики

Содержание

1. Макросы
    1. Генерация кода
    2. Синтаксические кавычки
    3. Пример: JS-подобные объекты
    4. Пример: разбор текста
2. Кодирование Чёрча
    1. Булева алгебра
    2. Пары и списки
    3. Персистентная память
3. Числа Чёрча
    1. Натуральные числа
    2. Арифметика
    3. Предикаты сравнения
    4. Числа со знаком
    5. Дальнейшие обобщения

Библиография

1. *Daniel Higginbotham* Clojure for the Brave and True (Chapter 8)
2. *Yehonathan Sharvit* Numbers and Arithmetics with functions only
3. *Yehonathan Sharvit* Boolean Algebra

Часть 4. Введение в Prolog

Тем: 3

Тема 12. Факты, правила и вычисления

Содержание

1. Факты и правила
    1. Термы
        1. Атомы
        2. Числа
        3. Структуры
    2. Факты
        1. Объявление
        2. Переменные
        3. Запросы
    3. Правила
        1. Объявление
            1. Простые
            2. Конъюнкция
            3. Дизъюнкция
            4. Отрицание
            5. Рекурсия
        2. Запросы
            1. Унификация
            2. Порядок выполнения
            3. Возвраты и отсечения
2. Вычисления
    1. Операторы и функции
    2. Рекурсивные вычисления
    3. Мемоизация
    4. Обратимые вычисления
3. Списки
    1. Синтаксис
    2. Сопоставление с образцом
    3. Правила на списках
    4. Правила высшего порядка

Домашнее задание 13. Простые числа на Prolog

1. Разработайте правила:
    - `prime(N)`, проверяющее, что `N` – простое число.
    - `composite(N)`, проверяющее, что `N` – составное число.
    - `prime_divisors(N, Divisors)`, проверяющее, что список `Divisors` содержит все простые делители числа `N`, упорядоченные по возрастанию. Если `N` делится на простое число `P` несколько раз, то `Divisors` должен содержать соответствующее число копий `P`. Например, `prime_divisors(12, [2, 2, 3])`.
2. Варианты
    - Простой: $N \le 1000$.
    - Сложный: $N \le 10^5$.
    - Бонусный: $N \le 10^7$.
3. До первого запроса будет выполнено правило `init(MAX_N)`.

Библиография

1. Adventure in Prolog
2. Wikibooks: Prolog
3. tuProlog Home
4. tuProlog Guide
5. SWI-Prolog

Тема 13. Задачи, унификация и объекты

Содержание

1. Задачи
    1. Задача о расстановке ферзей
    2. Порядок вычислений
    3. Загадка Эйнштейна
2. Унификация и объекты
    1. Унификация
    2. Определение
    3. Наследование
    4. Мультиметоды

Домашнее задание 14. Деревья поиска на Prolog

1. Реализуйте ассоциативный массив (map) на основе деревьев поиска. Для решения можно реализовать любое дерево поиска логарифмической высоты.
2. **Простой вариант.** Разработайте правила:
    - `map_build(ListMap, TreeMap)`, строящее дерево из упорядоченного списка пар ключ-значение (O(*n*));
    - `map_get(TreeMap, Key, Value)`, проверяющее, что массив содержит заданную пару ключ-значение (O(log *n*)).
    - `map_remove(TreeMap, Key, Result)`, удаляющее отображение для ключа (O(log *n*), можно без перебалансировки);
3. **Сложный вариант.** Дополнительно разработайте правила:
    - `map_put(TreeMap, Key, Value, Result)`, добавляющее пару ключ-значение в массив, или заменяющее текущее значение для ключа (O(log *n*));
    - `map_build(ListMap, TreeMap)`, строящее дерево из **не**упорядоченного списка пар ключ-значение (O(*n* log *n*)).

Библиография

1. Задача о восьми ферзях
2. Загадка Эйнштейна
3. Unification
4. Martelli-Montanari Algorithm

Тема 14. Разбор текста

Содержание

1. Термы
    1. Преобразование в терм
    2. Обратное преобразование
2. Списки символов
    1. Преобразование в список
    2. Преобразование списка в атом
    3. Обратное преобразование
3. Грамматики
    1. Описание грамматики
    2. Преобразование в строку
    3. Обратное преобразование

Домашнее задание 15. Разбор выражений на Prolog

1. Доработайте правило `eval(Expression, Variables, Result)`, вычисляющее арифметические выражения.
    1. Пример вычисления выражения `2(-x) - 3` для `x = 5`:

```
evaluate(
    operation(op_subtract,
        operation(op_multiply,
            const(2),
            operation(op_negate, variable(x))
        ),
        const(3)
    ),
    [(x, 5)],
    -13
)
```

    2. Поддерживаемые операции: сложение (`op_add`, `+`), вычитание (`op_subtract`, `-`), умножение (`op_multiply`, `*`), деление (`op_divide`, `/`), противоположное число (`op_negate`, `negate`).
2. **Простой вариант.** Реализуйте правило `postfix_str(Expression, Atom)`, разбирающее/выводящее выражения, записанные в постфиксной форме. Например,

```
postfix_str(
    operation(op_subtract,
        operation(op_multiply,
            const(2),
            operation(op_negate, variable(x))),
        const(3)
    ),
    '((2 (x negate) *) 3 -)'
)
```

3. **Сложный вариант.** Реализуйте правило `infix_str(Expression, Atom)`, разбирающее/выводящее выражения, записанные в полноскобочной инфиксной форме. Например,

```
infix_str(
    operation(op_subtract,
        operation(op_multiply,
            const(2),
            operation(op_negate, variable(x))),
        const(3)
    ),
    '((2 * negate x) - 3)'
)
```

4. Добавьте поддержку произвольного числа пробельных символов.
5. Правила должны быть реализованы с применением DC-грамматик.

Библиография

1. Prolog DCG Primer
2. Definite clause grammar
3. Definite Clause Grammars

- Часть 1. Объектно-ориентированное программирование
  - Тема 1. Программирование по контракту
    - Содержание
    - Слайды
    - Примеры
    - Домашнее задание 1. Обработка ошибок
    - Домашнее задание 2. Бинарный поиск
    - Библиография
  - Тема 2. Реализация ООП
    - Содержание
    - Слайды
    - Домашнее задание 3. Очередь на массиве
    - Домашнее задание 4. Очереди
    - Примеры
    - Экзаменационные вопросы
    - Практические навыки
  - Тема 3. Generics
    - Содержание
    - Слайды
    - Домашнее задание 5. Вычисление в различных типах
    - Библиография
- Часть 2. Введение в JavaScript
  - Тема 4. Функции
    - Содержание
    - Домашнее задание 6. Функциональные выражения на JavaScript
    - Библиография
  - Тема 5. Объекты и замыкания
    - Содержание
    - Домашнее задание 7. Объектные выражения на JavaScript
    - Библиография
  - Тема 6. Что еще есть в JavaScript
    - Содержание
    - Домашнее задание 8. Обработка ошибок на JavaScript
    - Библиография
- Часть 3. Введение в Clojure
  - Тема 7. Функции
    - Содержание
    - Домашнее задание 9. Линейная алгебра на Clojure
    - Code Golf (бонус)
    - Библиография
  - Тема 8. Внешний мир
    - Содержание
    - Домашнее задание 10. Функциональные выражения на Clojure
    - Библиография
  - Тема 9. Объекты
    - Содержание
    - Домашнее задание 11. Объектные выражения на Clojure
    - Библиография
  - Тема 10. Комбинаторные парсеры
    - Содержание
    - Домашнее задание 12. Комбинаторные парсеры
    - Библиография
  - Тема 11. Макросы и основания информатики
    - Содержание
    - Библиография
- Часть 4. Введение в Prolog
  - Тема 12. Факты, правила и вычисления
    - Содержание
    - Домашнее задание 13. Простые числа на Prolog
    - Библиография
  - Тема 13. Задачи, унификация и объекты
    - Содержание
    - Домашнее задание 14. Деревья поиска на Prolog
    - Библиография
  - Тема 14. Разбор текста
    - Содержание
    - Домашнее задание 15. Разбор выражений на Prolog
    - Библиография